1 The Twin

1.1 Role and goals

The Twin is the city. It simulates 42 field devices across six zones of Greater Tunis (Downtown Financial, Port & Coastal Harbor in La Goulette, North Industrial Park, West Residential Hills, Central University & Med, South Logistics Hub). Every zone has the same seven devices: two traffic cameras, two water-level floats, one power-load meter, one air-quality station and one emergency-call counter. Each device reports ten times per second, so the city streams about 420 signed readings per second whether or not anything is happening.

The Twin answers one question for its partners: *what is happening in the city right now?* It has three jobs:

1. **Stream telemetry.** Publish every reading to Kafka, signed with the device's own Ed25519 key, in a fixed and documented format of at most 300 bytes.
2. **Accept commands.** Expose the city's actuators (dispatch units, traffic signal plans, drainage valves, grid switching) and its containment controls (isolate, quarantine, reinstate, release, roll back a reading) as 16 MCP tools. A command changes the simulated physics, so a good decision visibly helps the city and a bad one does not.
3. **Be attackable, and keep the truth.** An adversary engine can spoof, replay, delay, corrupt or drop readings, plant a rogue device, steal a device key, freeze a sensor or make an actuator lie. The real physical state goes to a separate ground-truth ledger that no partner can read, so every scenario can be scored afterwards against what really happened.

Goals we hold ourselves to: a partner connects without reading our code (a one-page interface card, settings only, no hard-coded addresses); every scenario starts with one command and leaves a log; the same seed always gives the same physics; nothing a partner sends can crash the Twin or leak the ground truth. *Scope:* physics, incidents, attacks, signing, the Kafka stream, the MCP command surface, the ground-truth ledger, the operations console and the scenario runner. Out of scope: detecting incidents and choosing responses (Brain), judging whether data can be trusted (Guardian).

1.2 Architecture

Figure 1 shows the Twin as it runs: one Python process with two network faces, Kafka going out and MCP coming in, plus a separate red-team port that partners never see.

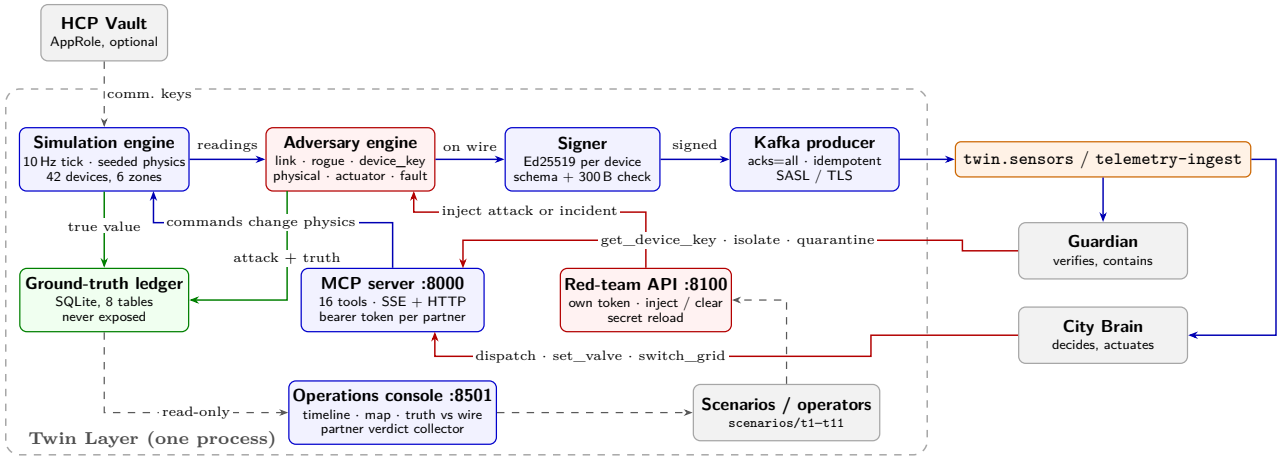


Figure 1: Twin architecture. Physics runs first and its true value goes to the ledger; the adversary engine may change what reaches the wire; every outgoing reading is schema-checked, size-checked and signed. Partners talk to the Twin only through Kafka (out) and MCP (in).

Two clocks. Simulation time drives all physics and every timestamp a partner sees. Wall-clock time only paces the loop: each tick advances the city by 0.1 s × `SIM_SPEED`, so scenarios can run 20 times faster without changing a single trajectory. Wall-clock time never appears in a payload, a ledger reading or an MCP reply.

Physics. Each sensor type follows its own model, integrated with a fixed step:

- *Traffic* follows a two-peak daily demand curve (about 120 veh/min at 08:30 and 135 at 17:30), with a fixed per-camera factor and first-order lag. Signal plans scale it (green wave ×1.15, red restriction ×0.55, all red ×0.05).
- *Water level* is a tank equation, $\dot{H} = Q_{in} - k\,c\sqrt{2g\,\max(0, H - H_0)}$, with a 12.4-hour tidal inflow, a per-device catchment factor and the valve opening k as the actuator. A ×3 storm with the valve half open crosses the 8 m danger line in about 17 simulated minutes; with the valve fully open it settles near 6.4 m.
- *Power load* peaks at 100 MW at 13:00; grid states scale it (`grid_b` ×0.85, `island` ×0.60, `shed` ×0.40). Above 114 MW for 60 s the zone blacks out.
- *Air quality* is zone background + traffic share + an industrial term + incident terms (a fire adds up to 150 AQI). *Emergency calls* are a Poisson process whose rate rises with jams, floods and incidents and falls when units are dispatched.

All randomness comes from one generator seeded by `SIM_SEED`, so the same seed gives the same city.

Incidents and attacks are kept apart. *Incidents* (rain storm, traffic accident, fire, grid overload, air hazard) change the real physics and end only when the right command arrives: a fire ends when a fire unit is dispatched, an overload when the zone sheds load. *Attacks* change only what goes on the wire. The ledger stores the real value next to what was broadcast, which is how a scenario proves, for example, that a "flood" reading was fake.

Ground-truth ledger. One SQLite file with eight tables (`runs`, `devices`, `readings_truth`, `attacks_injected`, `commands_log`, `containment_log`, `physical_events`, `secret_rotations`). Every command is logged with its caller, its arguments, whether it ran and, if not, why. Containment survives restarts.

1.3 Tech stack and why

Table 1: Twin technology stack

Component	Technology	Why we chose it
Language	Python 3.11	Same language as both partners; quick to change; mature crypto and Kafka libraries.
Telemetry bus	Apache Kafka 4.3 (KRaft), `confluent-kafka` producer	Durable, partitioned stream; librdkafka gives idempotent delivery, SASL and TLS out of the box. Readings are keyed by `device_id`, so each device stays in order.
Command interface	Model Context Protocol (official Python SDK), SSE and Streamable HTTP	A self-describing tool protocol: a partner, or an AI agent, can list our tools and their arguments without reading our code.
Signing	Ed25519 (`cryptography`)	64-byte signatures fit the 300-byte payload limit and are fast enough for 420 signatures per second.
Validation	Pydantic	One schema object rejects anything malformed before it is signed or sent.
Ground truth	SQLite (WAL mode)	No setup, one file, transactional, easy to inspect after a run; the console reads it read-only.
Red-team API	Starlette + Uvicorn on its own port	Keeps attack controls physically apart from the partner-facing MCP port.
Console	Streamlit, pydeck, Altair	Live timeline, map, truth-vs-broadcast curves and ledger browser with little UI code.
Packaging	Docker Compose	One command starts Kafka, the Twin and the console; every partner address and secret is an environment variable.
Testing	pytest	251 automated tests, a frozen contract snapshot, and 11 end-to-end scenarios with stand-in Guardian and Brain.

1.4 Interfaces in/out

Table 2: What the Twin sends and accepts

Dir.	Peer	Channel	Format	Security
Out	Guardian, Brain	Kafka `twin.sensors` by default (`telemetry-ingest` on the Guardian cluster), keyed by `device_id`	JSON $\leq$ 300 B: `reading_id, device_id, device_type, ts, seq, value, unit, location, status, sig`	Ed25519 over sorted compact JSON without `sig`; SASL PLAIN/SCRAM and TLS supported
In	Brain, Guardian	MCP on port 8000 (`/sse`, `/mcp`)	16 JSON-RPC tools: 6 queries, 5 actuation, 5 containment	Optional bearer token per partner; no token gives HTTP 401; caller recorded in the ledger
Out	Guardian	MCP `get_device_key`	Base64 Ed25519 public key; `null` for unknown devices	As above
In	Operators, scenarios	Red-team REST API on port 8100	Inject or clear incidents and attacks, status, journal, secret reload	Header `X-Red-Team-Token`; never shared with partners
In	Vault (optional)	HTTPS to HCP Vault, AppRole	The Twin's four communication keys, refreshed every 300 s	Certificate verified; tokens and keys never logged
In	Guardian or Brain (optional)	Kafka verdict topic, read by the console's verdict collector	Any JSON verdict naming a device or a reading; field names matched loosely	Read-only SASL user; placed on simulation time from our own ledger, never from the partner's clock

Example reading:

```
{"reading_id":"r-z1-tc1-7559","device_id":"z1-tc1","device_type":"traffic_camera","ts":"2026-10-17T16:11:56Z",
 "seq":7559,"value":87.54,"unit":"veh/min","location":"zone-1","status":"ok","sig":"gnSioLUV68nf...BrBw=="}
```

Example command and reply:

```
set_valve {"device_id":"z1-wl1","target_state":"open"} → {"status":"ok","device_id":"z1-wl1","valve_position":"open"}
```

Errors are always structured (for example `"code":"DEVICE_NOT_FOUND"`), and repeating a command with the same `idempotency_key` returns the cached result instead of acting twice.

Built to connect to anyone. Every partner setting is configuration, not code: broker list, topic, security protocol, SASL user and password file, CA file, acks, network name, host ports and Vault settings. We tested this on the Guardian's real three-broker SASL cluster, configuring our container through environment variables only. In a full three-layer run the Twin delivered 779,325 of 779,360 readings to `telemetry-ingest`; the 19 delivery failures all came at startup while the brokers were still electing coordinators. Against the Guardian's final version it delivered 126,344 of 126,378, the Guardian's metadata producer listed all 42 devices and 6 zones through our MCP tools, and its validation job read our stream. The Brain consumed more than 300,000 of our readings, rejected the tampered ones itself (690 bad signatures, 352 frames from an unregistered device) and executed its valve commands through MCP. The Guardian's own verifier accepted 3,000 of 3,000 of our signatures. The 16 tools and the reading schema are frozen by a contract test, so no change can break a partner silently.

1.5 Security measures

We work on three fronts: (a) we give the other layers realistic attacks to catch, (b) we protect the Twin and its partners, and (c) we prove it with tests and scenario logs.

1.5.1 a) Threats we simulate

Table 3: The adversary engine: six attack capabilities, each injected live and recorded with the truth

Capability	What the attacker does on the wire	What should catch it
`device_physical`	Fakes a sensor value; the signature is still valid because the real device signs it	Guardian: two floats in the same zone disagree
`device_key`	Sends readings signed with a stolen key, e.g. 180 MW on a 120 MW meter	Guardian: physically impossible value; quarantine
`link`	Corrupts bytes, breaks the signature, replays a `reading_id`, delays frames by 5 s, or drops them	Guardian: bad signature, duplicate ID, late timestamp, silence
`rogue`	Sends as an unregistered device	Guardian: `get_device_key` returns `null`; quarantine
`actuator`	Answers "ok" to a command but does nothing	Brain: the readings do not move toward the commanded state
`sensor_fault`	Drift, stuck value or zero dropout	Guardian: statistics (stuck or drifting signal)

1.5.2 b) How the Twin protects itself and its partners

- **Signed telemetry.** Every reading is signed with its device's Ed25519 key; public keys come only from MCP `get_device_key`. Keys and sequence numbers persist across restarts, so a restart never looks like an attack.
- **Containment is enforced, not just recorded.** Quarantined and isolated devices keep reporting, with that status in every reading, so the Brain never loses sight of them; they refuse actuation with `DEVICE_CONTAINED`; a quarantined rogue identity is blocked from the wire. Containment survives restarts.
- **Authenticated commands.** Optional bearer tokens per partner, read from `MCP_AUTH_TOKENS_FILE` by the Twin, the scenarios and the mocks alike; a call without a valid token gets HTTP 401. Every command is attributed to its caller, so the timeline shows whether the Brain, the Guardian or the test harness acted.
- **The ground truth is out of reach.** No MCP tool reads the ledger or the real physical state: queries return the last broadcast value, exactly what a partner would have seen on Kafka.
- **Attack controls are separate.** The red-team API runs on its own port behind its own token.
- **Strict input checks.** Unknown devices, unknown target states and malformed arguments return structured errors. Every outgoing payload is schema-checked and size-checked before signing.
- **Secrets handling.** The Kafka password, MCP tokens and red-team token come from a pluggable hook, a file (Docker, Kubernetes or Vault Agent style) or the environment, never from code. They rotate without a restart (`POST /api/secrets/reload` or `SIGHUP`), and a new Kafka password is tested against the broker before the producer switches. The Twin can also fetch its communication keys from the triplet's HCP Vault with its own AppRole (read-only on its four paths; verified live, other teams' keys refused).
- **Durable delivery.** `acks=all` with idempotence, so a retry never duplicates a reading.

1.5.3 c) Evidence

251 automated tests cover signature round-trips, payload limits, containment, secret rotation, compose configuration, the one-page interface card and a frozen contract snapshot. The 11 end-to-end scenarios (Table 4) each start with one command and write a log to `logs/`; all 11 pass back to back on the same running Twin, also with MCP authentication switched on. A scenario with a wrong token, an unreachable Twin or an unreadable stream stops with a clear reason instead of hanging or checking nothing.

Table 4: Twin scenarios (`scenarios/t1`–`t11`); each passes with the stand-in Guardian and Brain

ID	Scenario	What the Twin does and what the log proves
T1	Calm baseline	42/42 devices emit, every frame $\leq$ 300 B, every signature verifies, about 420 msg/s
T2–T5	Accident, port flood, fire, grid overload	Real physics changes; each incident ends only after the matching command (a valve opened in time keeps the flood below 8 m, observed peak 7.25 m)
T6	Fake port flood	`z2-wl1` broadcasts 9.5 m with a valid signature while the true level stays below 5 m
T7–T9	Stuck sensor, rogue device, stolen key	Frozen value, unregistered sender, 180 MW signed with a real key; each is recorded with the truth
T10	Link tampering	Corrupt bytes, broken signature, replay, delay (measured lag 5.00 s) and drop on `z3-tc1`
T11	Storm + lying actuator	`set_valve open` answers "ok" but the ledger records `executed: false` and the water keeps rising

1.6 Known limitations

1. **MCP and the red-team API run over plain HTTP.** Bearer tokens control who may call, not who can listen. *Mitigation:* a TLS reverse proxy in front for any non-local deployment.
2. **Authentication depends on the partner.** Tokens are optional so that a partner without token support can still connect. In our runs the Guardian's MCP client sent no token, so authentication stayed off and its calls were logged as anonymous. *Mitigation:* every partner sends its own token; the Twin, scenarios and mocks already read `MCP_AUTH_TOKENS_FILE`.
3. **Device signing keys live in a local keystore file and do not rotate.** Rotating them would break every partner's verification. *Mitigation:* move them to Vault with a versioned key-ID scheme agreed by all three layers.
4. **Vault keys are fetched but not used.** The triplet has not agreed what to encrypt with them, so no traffic is encrypted yet.
5. **Simplified physics.** Each water sensor is a single tank, power is one meter per zone, and incidents have fixed effects. The models behave plausibly (rush hours, tides, a midday peak) but are not fitted to measured data from Tunis.

6. **One process, one machine.** No high availability. With three Kafka brokers and all layers on one laptop, throughput fell to about 300 readings per second; with the Guardian's full processing stack on the same laptop it fell to 22–68 readings per second and MCP replies took 10–25 s, because the CPU was saturated. *Mitigation:* each layer on its own machine for the demo.
7. **Units are frozen as strings.** Water is reported as `"meters"`; a partner expecting `"m"` must map it. We kept it so existing consumers do not break.
8. **The ledger grows without limit** (about 127 MB after long runs), and **the console needs internet** for its map tiles and fonts. *Mitigation:* prune the ledger per run; bring a hotspot or bundle the assets offline.